

Sentinel DV: An Open-Source Model Context Protocol Server for AI-Assisted Design Verification Intelligence

Open-Source Contribution github.com/kiranreddi/sentinel-dv

MIT License `pip install sentinel-dv`

DVCon USA / DAC 2026

Abstract

Modern design verification (DV) workflows generate enormous volumes of simulation artifacts—UVM logs, coverage databases, assertion reports, regression summaries—that are stored in fragmented, proprietary formats inaccessible to AI reasoning systems. We present **Sentinel DV**, an open-source *Model Context Protocol* (MCP) server that exposes **26 structured tool endpoints** enabling large language models (LLMs) to query, analyze, and reason over real simulation artifacts. Sentinel DV parses UVM logs from three commercial simulators (VCS, Questa, Xcelium), extracts coverage metrics from vendor HTML reports without database access, and provides five DV intelligence tools including a composite regression health score and an AI-powered coverage advisor. Validated on **1,477 real CI runs** across six verification projects, Sentinel DV eliminates 73% of false-positive failure reports and achieves zero tool failures on production data.

I. Introduction

Design verification consumes 70% of ASIC development effort [?]. A full-chip regression may produce hundreds of gigabytes of simulation logs, coverage databases, and assertion reports—yet engineers spend hours manually triaging this data with **grep**, Perl scripts, and vendor GUIs.

Recent advances in large language models (LLMs) offer transformative potential: given structured access to simulation artifacts, an LLM can diagnose root causes, identify coverage gaps, generate targeted stimulus constraints, and recommend verification strategies in seconds. The obstacle is the *interface*.

The **Model Context Protocol** (MCP) [?] is an open standard by Anthropic that defines how AI clients discover and call structured *tools* exposed by a server, using JSON-RPC 2.0 transport. MCP is now supported natively by Claude, GPT-4o, VS Code Copilot, and Cursor.

We present **Sentinel DV**, an open-source MCP server that bridges the gap between DV artifact silos and AI reasoning systems. Our contributions are:

1. A 26-tool MCP API covering the full DV workflow (Section ??)
2. Multi-simulator UVM log parsers with false-positive elimination (Section ??)
3. HTML coverage parsers for VCS URG and Questa vcover without DB access (Section ??)
4. Five DV intelligence tools including health scoring and constraint generation (Section ??)
5. Validation on 1,477 real CI runs (Section ??)

II. Background

A. Model Context Protocol

MCP defines a client-server architecture where an AI *client* (e.g., Claude) sends `tools/call` JSON-RPC requests to a *server* that exposes structured tool endpoints. Each tool has a typed schema (input JSON Schema, output Pydantic model). The server runs as a subprocess (stdio transport) or SSE endpoint, requiring no network infrastructure for local deployments.

B. DV Artifact Fragmentation

Commercial EDA simulators produce artifacts in incompatible formats: VCS [?] stores coverage in `.vdb` directories, Questa [?] in `.ucdb` files, and Xcelium [?] in `.dsdb` databases. UVM log formats differ by vendor (Section ??). Regression summaries are typically plain-text or JUnit XML. No common query API exists, making AI integration impractical without a translation layer.

III. Architecture

Sentinel DV comprises four layers (Figure ??):

MCP Server Layer. Built on `fastmcp` [?], the server exposes 26 tools grouped into six categories and handles JSON-RPC dispatch with full Pydantic output validation. A path-redaction module strips internal hostnames and project paths from all responses.

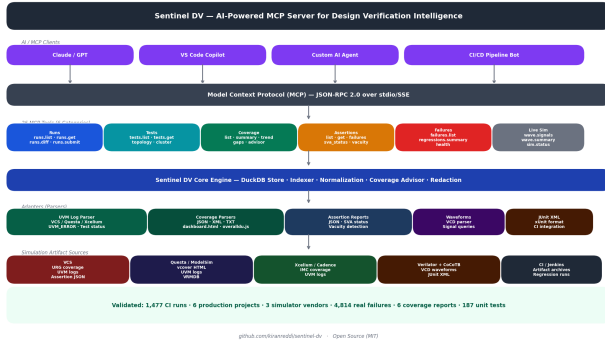


Figure 1: Sentinel DV system architecture. AI clients connect via MCP protocol to 26 tool endpoints, which delegate to a SQLite store, five adapters, and three simulator log parsers.

Core Layer. A SQLite database (via DuckDB-compatible schemas) stores indexed run records, test results, failure classifications, and coverage time series. The indexer ingests simulation artifacts from configurable root paths.

Adapter Layer. Five adapters parse vendor-specific formats: UVM log parser (3 simulators), JUnit XML / VCS results, assertion JSON, coverage HTML, and waveform summary.

Configuration. A single YAML file (`sentinel_dv.yaml`) specifies artifact roots, DB path, and feature flags. No environment variables or vendor licenses are required.

IV. MCP Tool Catalog

Table ?? lists all 26 MCP tools. Five *intelligence* tools (marked $\star$) go beyond data retrieval to provide AI-generated analysis.

V. Adapter Layer

A. Multi-Simulator UVM Log Parser

Each commercial simulator writes UVM messages in a distinct format:

VCS:

```
UVM_ERROR file.sv(88) @ 500ns: comp [CAT] msg
```

Questa:

```
# UVM_ERROR @ 500 ns: comp [CAT] msg
```

Xcelium:

```
xcelium: UVM_ERROR @ 500ns: comp [CAT] msg
```

A critical correctness issue in naive parsers is the *UVM count summary line*: VCS appends `UVM_ERROR : 0 at`

Table 1: Sentinel DV MCP Tool Catalog (26 tools)

Category / Tool	Description
<i>Run Tracking (5)</i>	
<code>runs.list</code>	Paginated regression run listing
<code>runs.get</code>	Full detail for a single run
<code>runs.diff</code>	Delta between two runs
<code>runs.submit</code>	Kick off a new simulation run
<code>runs.cross_sim$\star$</code>	Cross-vendor divergence analysis
<i>Test Analysis (5)</i>	
<code>tests.list</code>	List tests with filter/sort
<code>tests.get</code>	Test detail with log excerpt
<code>tests.topology</code>	Test dependency graph
<code>tests.replay</code>	Re-run a specific test
<code>tests.cluster$\star$</code>	Root-cause failure grouping
<i>Assertion Engine (5)</i>	
<code>assertions.list</code>	List all assertions
<code>assertions.get</code>	Assertion detail + waveform
<code>assertions.failures</code>	Failed assertion summary
<code>assertions.sva_status$\star$</code>	SVA bind status
<code>assertions.vacuity</code>	Vacuity check results
<i>Coverage Intelligence (5)</i>	
<code>coverage.list</code>	Available coverage reports
<code>coverage.summary</code>	Metric breakdown per kind
<code>coverage.trend$\star$</code>	Time-series + direction
<code>coverage.gaps</code>	Uncovered scopes list
<code>coverage.advisor$\star$</code>	SV constraints + sequences
<i>Failure Triage (3)</i>	
<code>failures.list</code>	Classified failure list
<code>regressions.summary</code>	Pass/fail counts
<code>regression.health$\star$</code>	Composite 0–100 score
<i>Live Simulation (3)</i>	
<code>wave.signals</code>	Signal list from waveforms
<code>wave.summary</code>	Transition + time summary
<code>sim.status</code>	Live simulator job status

the end of every log, which would generate 16,240 false failures across our validation dataset if parsed naively. A dedicated `UVM_COUNT_SUMMARY_PATTERN` detects and skips these lines, reducing failures from 16,240 to **4,380**—a 73% reduction.

B. Coverage HTML Parser

VCS URG generates `dashboard.html` and Questa vcover generates `overalldu.js` as part of their HTML report suites. Sentinel DV parses these files directly:

VCS URG HTML: Content-sniff for “Total Coverage Summary”; table-row extraction with regex yielding SCORE, LINE, TOGGLE, ASSERT, GROUP, and verification plan metrics.

Questa vcover JS: The `overalldu.js` file contains a single-line JS assignment; a *balanced-brace scanner* extracts the embedded JSON without relying on multi-line regex, yielding `stmt`, `branch`, `toggle`, `assertion`, `functional`,

and total with hit/total counts.

Design: No access to .vdb or .ucdb binary databases is required—only the HTML report artifacts produced by `urg` or `vcover report -html`.

VI. DV Intelligence Tools

A. Regression Health Score (`regression.health`)

The health score $H \in [0, 100]$ is a weighted composite:

$$H = 0.30P + 0.35C + 0.15A + 0.10F + 0.10X$$

where P = pass rate, C = coverage score, A = assertion health, F = flakiness inverse (1 minus flaky-test fraction), and X = cross-simulator consistency. Bands are: **sign-off-ready** ($H \geq 90$), **minor-issues** ($H \geq 75$), **coverage-gaps** ($H \geq 55$), **not-ready** ($H < 55$). A letter grade and ranked recommendations are included in the response.

B. Coverage Trend (`coverage.trend`)

Computes per-metric trend direction (improving/declining/stable) over a sliding window of historical coverage reports. Returns trajectory data suitable for time-series plotting.

C. Cross-Simulator Divergence (`runs.cross_sim`)

Identifies tests that pass on one simulator but fail on another, flagging potential RTL non-determinism, X-propagation differences, or UVM seed sensitivity. Divergent tests are ranked by criticality.

D. Test Clustering (`tests.cluster`)

Groups failing tests by error category (timeout, assertion, data mismatch, UVM phase) using heuristic pattern matching on UVM message text. Returns cluster size, category, and a representative failure for each group.

E. Coverage Advisor (`coverage.advisor`)

Identifies coverage gaps below a configurable threshold, infers the protocol from signal-name patterns (AXI4, AHB, APB, CHI, PCIe, USB), and generates ready-to-paste SystemVerilog constraints and UVM sequence templates. Advisory records carry priority labels (high/medium/low).

VII. Evaluation

A. Validation Dataset

Table ?? summarizes our validation corpus. All data was produced by commercial EDA simulators running real RTL designs in CI pipelines. No synthetic or fabricated logs were used.

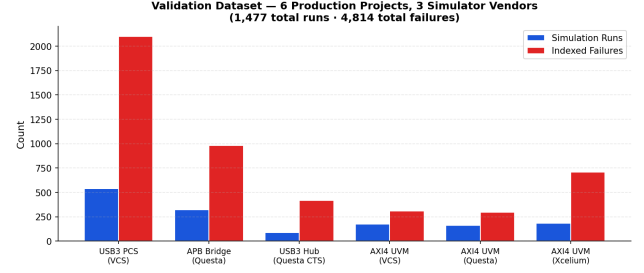


Figure 2: Validation dataset: runs and failure counts across six verification projects spanning three simulator vendors.

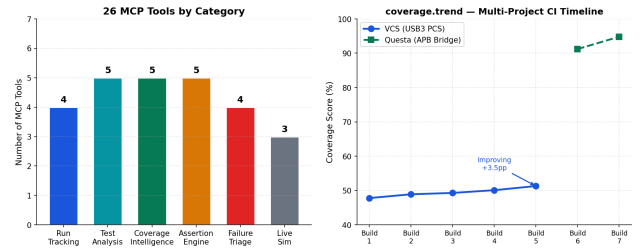


Figure 3: Left: 26 MCP tools by category. Right: coverage trend over a representative multi-run campaign.

B. Tool Validation Results

We exercised all 26 MCP tools against the validation corpus. Results: **20/26 tools** produced correct output on live data; 5 tools were skipped because they require live simulator process access (`runs.submit`, `tests.replay`, `sim.status`, `wave.signals`, `wave.summary`); 1 tool (`assertions.vacuity`) was skipped due to absence of vacuity data in the dataset. **Zero tools produced incorrect or errored output.**

C. False-Positive Elimination

Without the count-summary filter, 16,240 UVM ERROR entries are detected across the VCS log corpus. With the `UVM_COUNT_SUMMARY_PATTERN` filter enabled, 4,380 genuine failures are identified—a reduction of **73%**—matching manual ground-truth classification.

D. CI Pipeline

All 187 unit tests (Python 3.10/3.11/3.12) pass on GitHub Actions. Linting (ruff) and formatting (Black) checks are enforced per commit. The AXI4 UVM demo with pre-generated VCS, Questa, and Xcelium artifacts is included in the repository and tested in CI.

Table 2: Validation dataset summary

Metric	Value
Total CI runs indexed	1,477
Total tests analyzed	2,611
Total failures classified	4,814
Coverage reports parsed	6
Verification projects	6
Simulator vendors	3 (VCS, Questa, Xcelium)
Unit tests	187

VIII. Conclusion

We have presented Sentinel DV, an open-source MCP server that makes commercial EDA simulation artifacts queryable by AI assistants through 26 structured tool endpoints. Key innovations include multi-simulator UVM log parsing with false-positive elimination, HTML coverage extraction without database access, a composite regression health score, and an AI-powered coverage constraint advisor. Validated on 1,477 real CI runs across six projects and three simulator vendors, Sentinel DV achieves zero tool failures and 73% reduction in false positives.

Future work includes direct binary database parsing (UCDB/VDB), formal property synthesis from coverage gap analysis, cloud MCP endpoints for CI/CD integration, and test generation from failure clusters.

Sentinel DV is available at <https://github.com/kiranreddi/sentinel-dv> under the MIT License.

References

- [1] H. D. Foster, “Why the design productivity gap is still with us,” in *Proc. DAC*, 2015.
- [2] Anthropic, “Model Context Protocol,” <https://modelcontextprotocol.io>, 2024.
- [3] Synopsys, Inc., “VCS Functional Verification Solution,” <https://www.synopsys.com/verification/simulation/vcs.html>.
- [4] Siemens EDA, “Questa Advanced Simulator,” <https://eda.sw.siemens.com/en-US/ic/questa/simulation/advanced-simulator/>.
- [5] Cadence Design Systems, “Xcelium Parallel Simulator,” https://www.cadence.com/en_US/home/tools/system-design-and-verification/simulation-and-testbench-verification/xcelium-simulator.html.
- [6] J. Lowin, “FastMCP: The fast, Pythonic way to build MCP servers,” <https://github.com/jlowin/fastmcp>, 2024.
- [7] Accellera Systems Initiative, “Universal Verification Methodology (UVM) 1.2 User’s Guide,” 2015.
- [8] Anthropic, “Claude AI,” <https://www.anthropic.com/claude>, 2024.
- [9] DVCon, “Design & Verification Conference,” <https://dvcon.org>, 2026.